

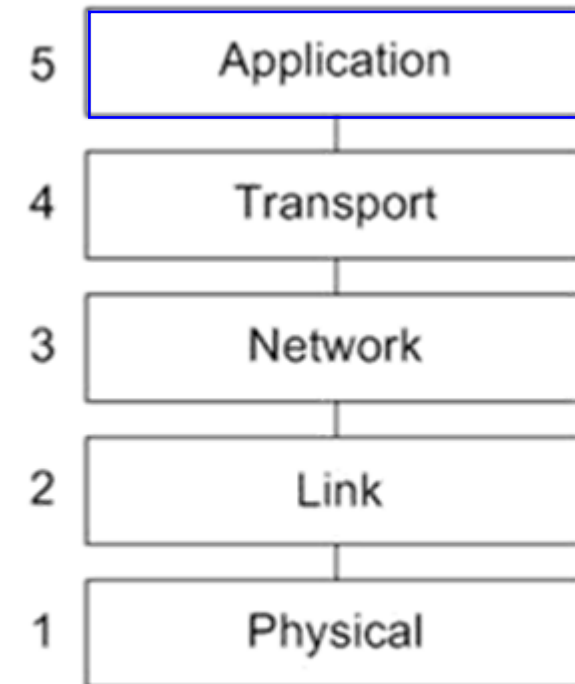
Networking

Les 1b

Application Layer en Protocols

Agenda

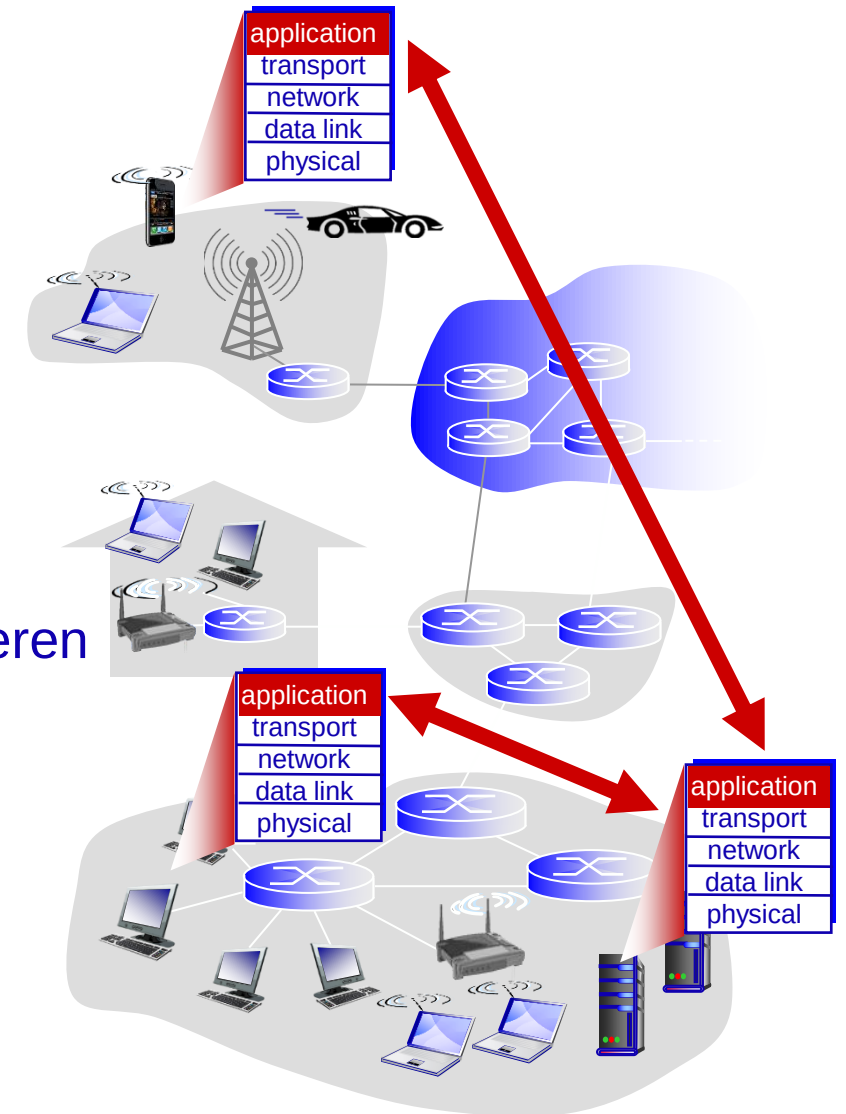
- Application Layer
- Application protocols
- Sockets en port nummers
- Application protocol voorbeelden
- Netwerkarchitecturen



TCP/IP Model

Application layer

- Programma's met netwerk-capaciteit
- Start- en eindpunt van een netwerksessie
- Bestaat uit programma's die met elkaar communiceren



Application protocols

- Applicatieprotocollen definiëren:
 - Het type berichten wat wordt uitgewisseld
 - E.g., request, response
 - Message syntax
 - Uit welke velden een bericht bestaat
 - Hoe velden van elkaar zijn gescheiden
 - Message semantics
 - Betekenis van informatie in velden

Open protocollen:

- gedefinieerd in RFCs
- staan interconnectiviteit toe
- e.g., HTTP, SMTP

Bedrijfseigen protocollen:

- e.g., Skype

Regels die dicteren wanneer en hoe processen berichten verzenden, en daarop reageren

Sockets en port numbers

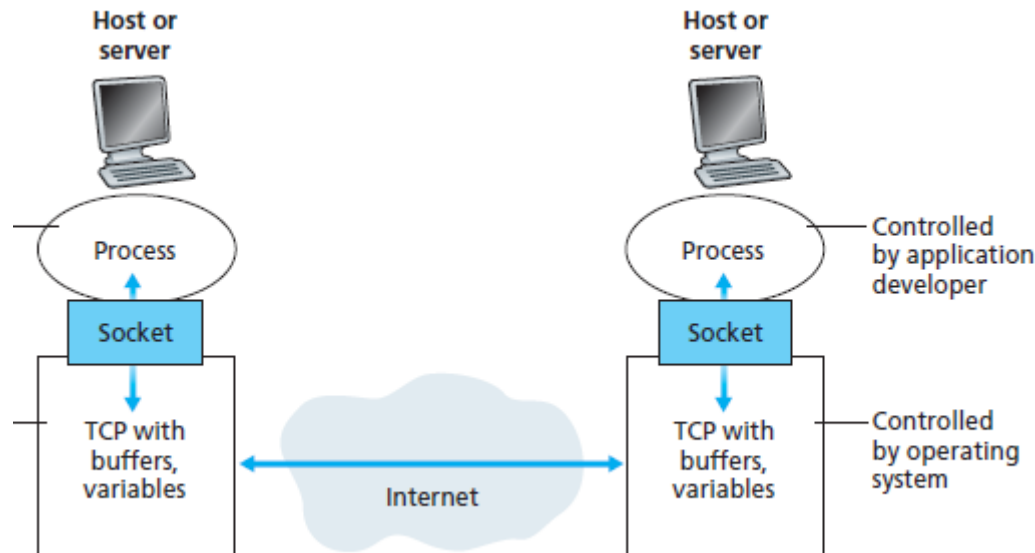


Figure 5

- Een applicatie communiceert met de transport layer via een **socket**
- Een applicatie identificeert zichzelf met een **port number**
- Veel populaire protocollen hebben bekende port numbers: port 80 for HTTP, port 25 for SMTP, etc.

Application protocols

- Er zijn veel verschillende application protocols...

Applicatie	Protocol	Port
E-mail	SMTP	25
Web	HTTP, HTTPS	80, 443
File transfer	FTP	20
Remote access	Telnet	23
Peer to peer	BitTorrent	6881-6889
Remote login	SSH	22

Web en HTTP

- *Allereerst, een review...*
- Een *web page* bestaat uit *objects*
- Objecten kunnen van alles zijn: HTML bestanden, een plaatje, javascript bestand of stylesheet, audio file, etc...
- Elke webpagina bestaat uit een *basis HTML-bestand* waarin verschillende *referenced objects* zijn gedefinieerd
- Elk object is adresseerbaar via een *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model
 - *client*: browser doet *requests*, ontvangt een antwoord, en "vertoont" Web objects
 - *server*: Web server verstuurt (via HTTP protocol) objects in *response op requests*

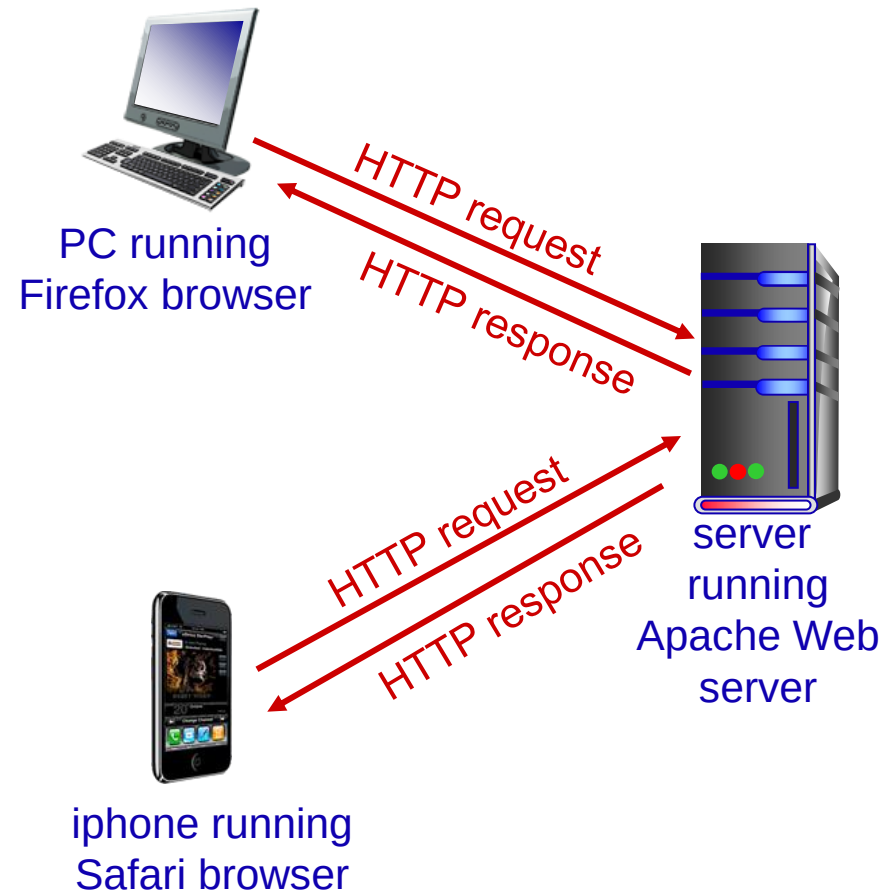


Figure 6

HTTP Requests

Een host kan verschillende soorten HTTP requests maken. Deze verschillende soorten noemen we *request methods*:

- GET
- POST
- PUT
- HEAD
- DELETE
- En nog wat meer...

We gebruiken voornamelijk:

- GET: om een object van de server op te vragen, zoals een pagina of plaatje
- POST: om een object van de server op te vragen, en tegelijkertijd informatie mee te sturen. Bijvoorbeeld: het invullen van een login-formulier, wat je doorstuurt naar je user page.

HTTP Protocol

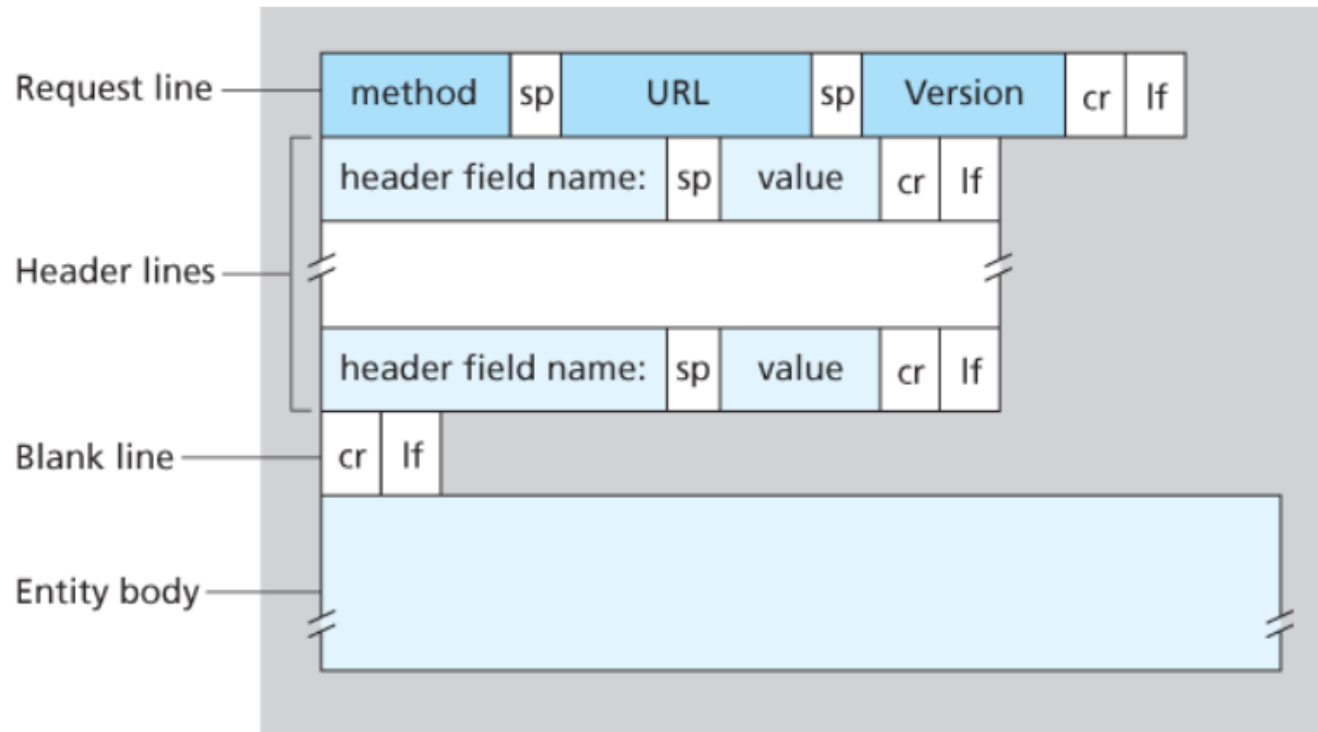


Figure 2.8 General format of an HTTP request message

HTTP Protocol

GET /somedir/page.html HTTP/1.1

Host: www.someschool.edu

Connection: close

User-agent: Mozilla/5.0

Accept-language: fr

HTTP Protocol

HTTP/1.1 200 OK

Connection: close

Date: Fri, 24 Sep 2021 12:40:00 GMT

Server: Apache/2.2.3 (CentOS)

Last-Modified: Fri, 24 Sep 2021 12:05:00 GMT

Content-Length: 9000

Content-Type: text/html

(data data data data data data data data...)

Electronic mail

Drie belangrijke componenten

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, Thunderbird, iPhone mail client
- outgoing, incoming messages stored on server

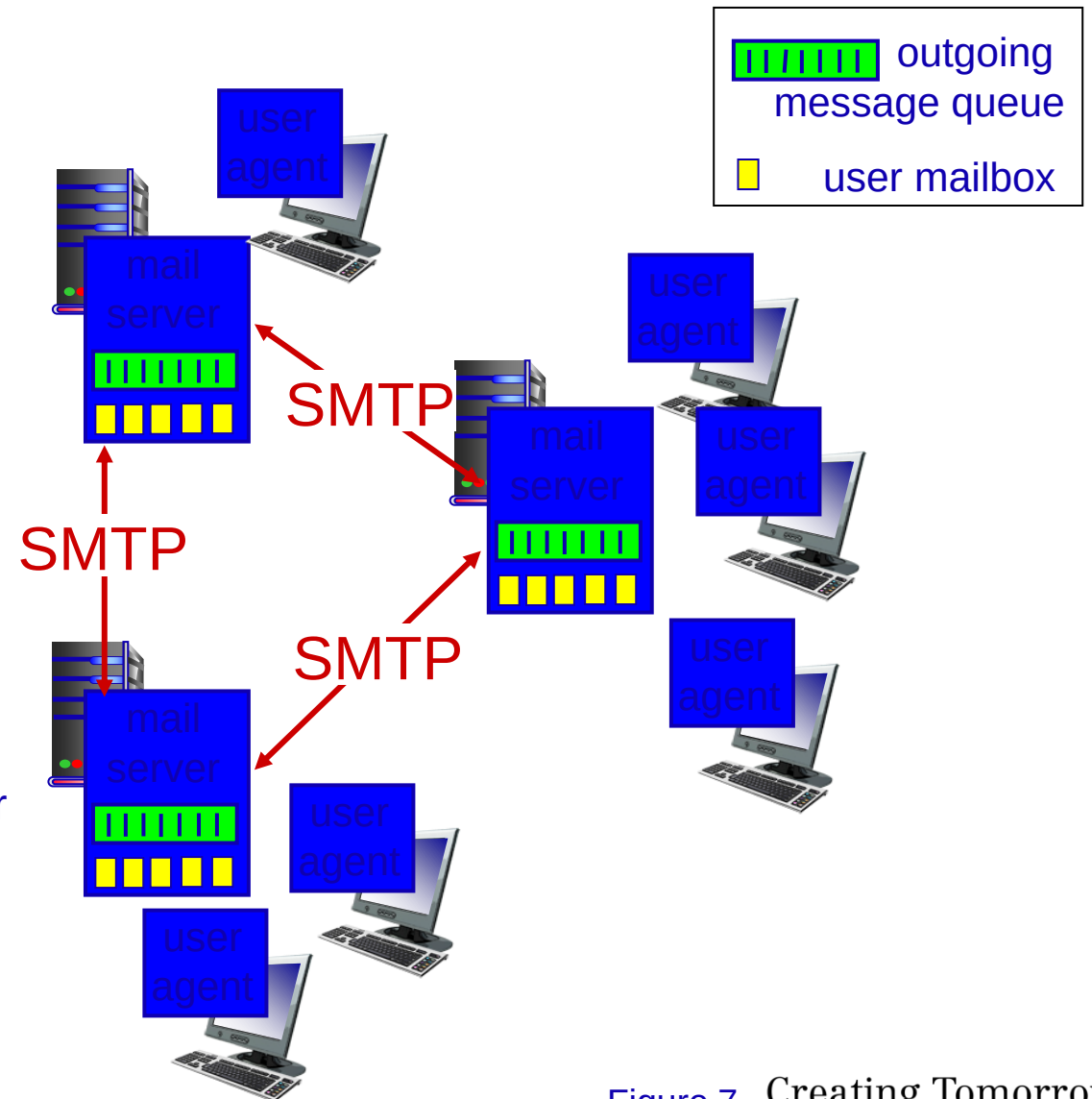
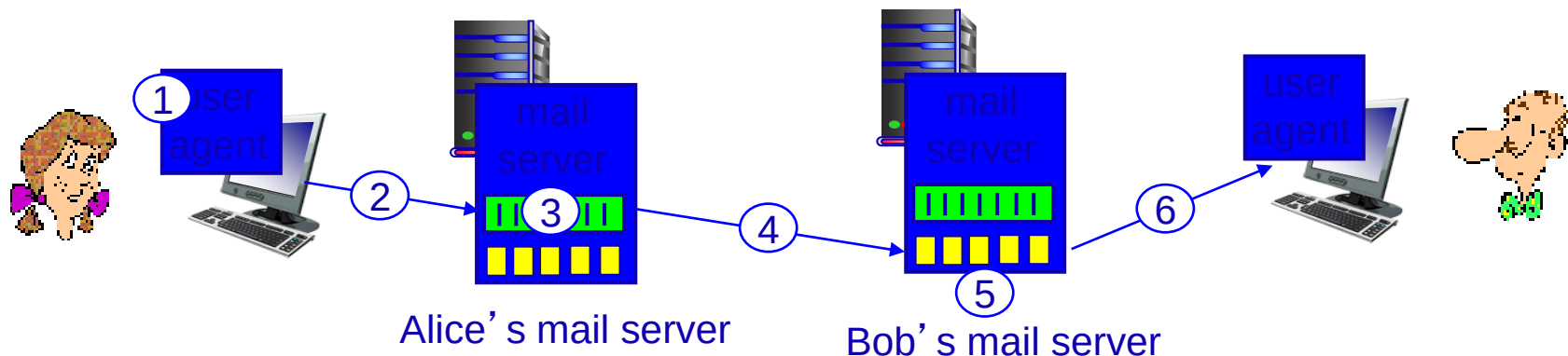


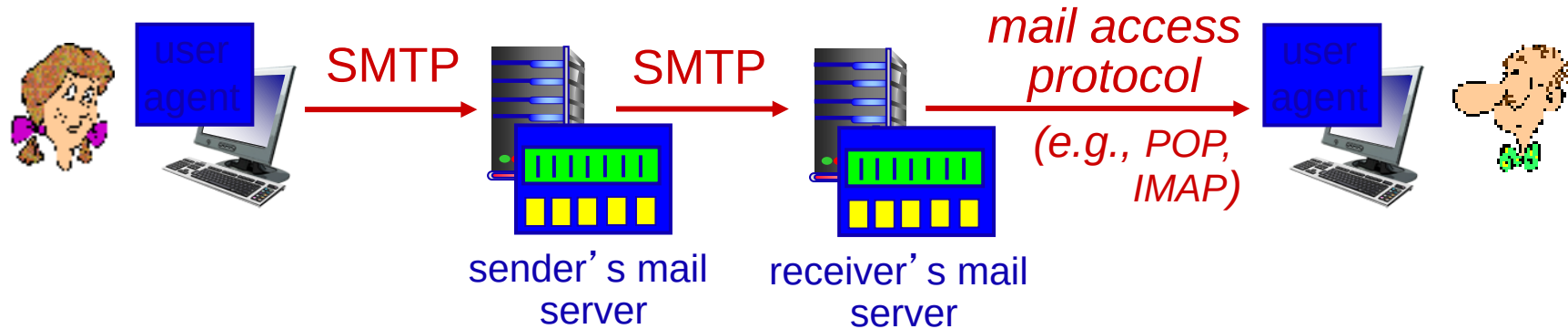
Figure 7 Creating Tomorrow

Scenario: Alice sends message to Bob

- 1) Alice stelt een bericht op “naar” bob@someschool.edu
- 2) Alice's email client verstuurt bericht naar haar mail server; bericht komt in message queue
- 3) client side van SMTP opent een verbinding met Bob's mail server
- 4) SMTP client verstuurt Alice's bericht via de TCP verbinding
- 5) Bob's mail server plaatst het bericht in Bob's mailbox
- 6) Bob opent zijn email client om het bericht te lezen



Mail access protocols



- **SMTP**: verzending/opslag naar receiver's server
- mail access protocol: ophalen van de server
 - **POP**: Post Office Protocol [RFC 1939]: authorization, download
 - **IMAP**: Internet Mail Access Protocol [RFC 1730]: more features, including manipulation of stored msgs on server
 - **HTTP**: gmail, Hotmail, Yahoo! Mail, etc.

POP3 (more) en IMAP

POP3

- Eerdere voorbeeld gebruikt POP3 “download and delete” modus
 - Bob kan de mail niet downloaden op twee email clients
- POP3 “download-and-keep”: bewaart kopieën van berichten tussen email clients
- POP3 is stateless tussen sessions

IMAP

- Bewaart alle berichten op één plek: de server
- Staat gebruiker toe om berichten te plaatsen in folders
- Behoudt user state gedurende sessions:
 - namen van folders en mappings tussen message IDs en folder naam

SMTP

```
S: 220 smtp.example.com ESMTP Postfix
C: HELO relay.example.org
S: 250 Hello relay.example.org, I am glad to meet you
C: MAIL FROM:<bob@example.org>
S: 250 Ok
C: RCPT TO:<alice@example.com>
S: 250 Ok
C: RCPT TO:<theboss@example.com>
S: 250 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: From: "Bob Example" <bob@example.org>
C: To: "Alice Example" <alice@example.com>
C: Cc: theboss@example.com
C: Date: Tue, 15 Jan 2008 16:02:43 -0500
C: Subject: Test message
C:
C: Hello Alice.
C: This is a test message with 5 header fields and 4 lines in the message body.
C: Your friend,
C: Bob
C: .
S: 250 Ok: queued as 12345
C: QUIT
S: 221 Bye
{The server closes the connection}
```

Application architectures

- Mogelijke architecturen van applicaties
- client-server
- peer-to-peer (P2P)

Client-server architecture

Client requests service from server

server:

- always-on host
- permanent IP address
- data centers for scaling

clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

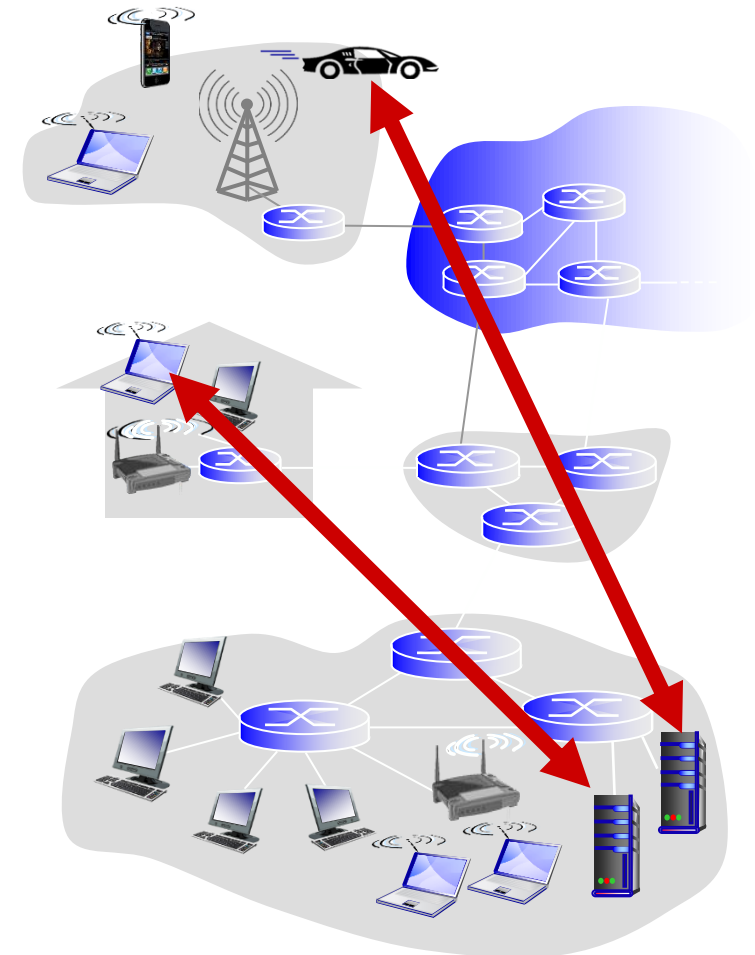


Figure 3

P2P architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management

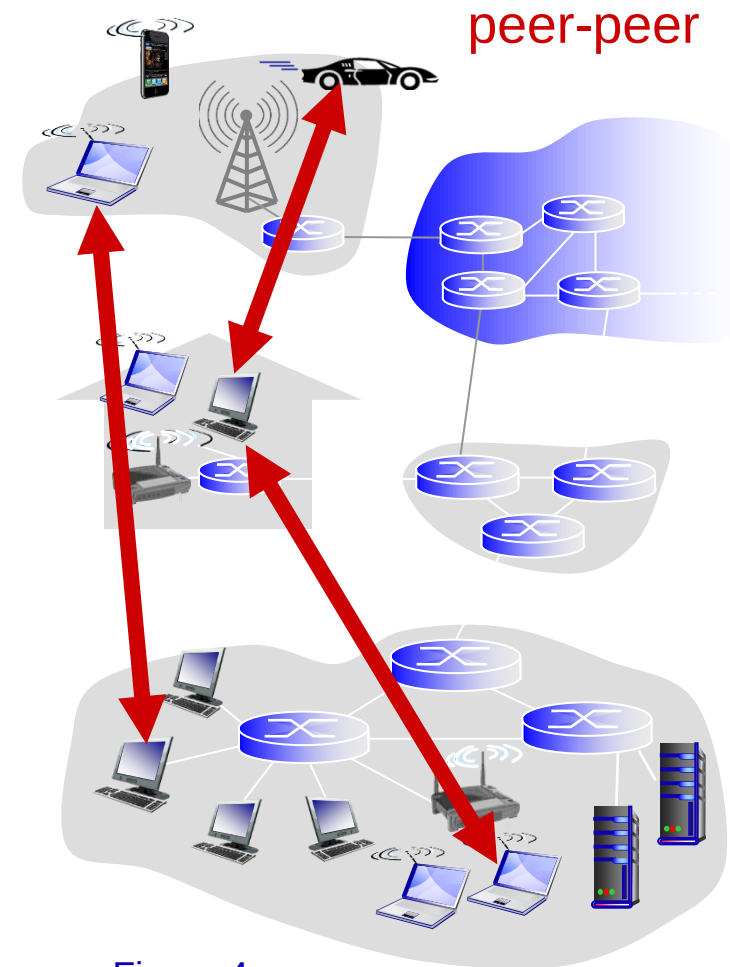


Figure 4